

Java microservices

The enterprise market has been traditionally dominated by Web architecture. The Java platform has been the automatic choice for developing enterprise Web applications because of the proven servlet technology, along with numerous frameworks ranging from the old Struts and JSF to the current day Spring. Figure 2 shows the architecture of a traditional Java Web application deployment.

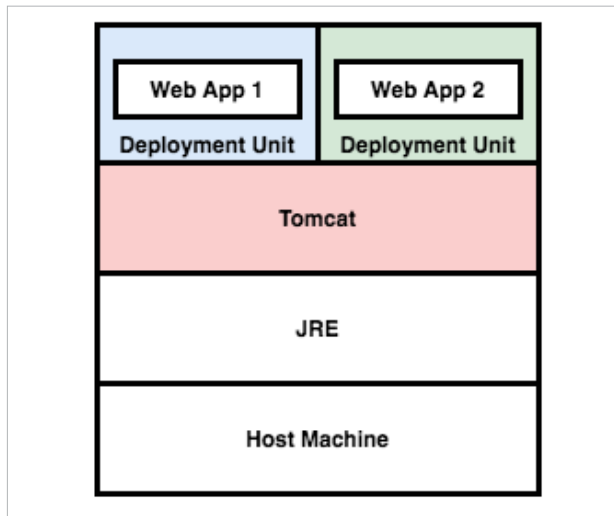


Figure 2: Deployment stack of traditional Java Web application

In the traditional approach, the Tomcat Web server had to be installed and provisioned before deploying the Web application. Moreover, multiple Web applications used to be deployed on a single Tomcat Web server. In other words, the deployment unit consists of just the application code. This approach is not compatible with the microservices architecture. Why?

Since microservices are expected to be deployed frequently (several times a day in some cases), the deployment process should be short, automatic, and smooth! This implies that the process of pre-installing and provisioning the Web server is clearly not an option anymore. On the other hand, as the microservices are independent of each other, co-hosting multiple services on the same Web server is also ruled out. Then, what is the way out?

Spring Boot framework

There are several frameworks available for building microservices on the Java platform including DropWizard and Vert.x. However, Spring Boot is the most popular framework in the community. It is built on top of the time-tested Spring framework and comes with all of the latter's proven features like dependency injection (IoC), aspect injection (AOP), Web context (MVC), JPA integration (ORM), etc. Naturally, it is very easy to adopt Spring Boot if you have worked with Spring earlier.

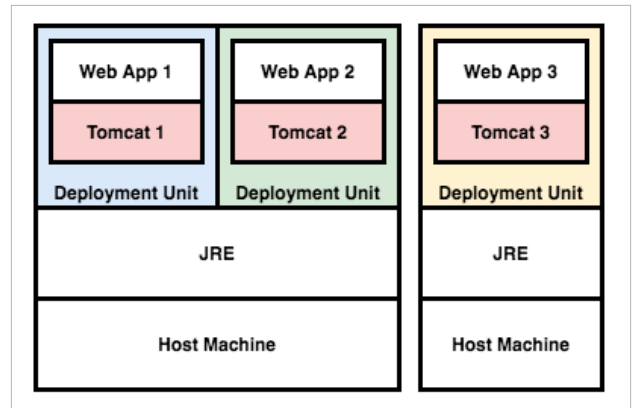


Figure 3: Deployment stack of the Spring Boot application

So, what is the value-add that Spring Boot offers? Spring Boot is an opinionated framework that prefers convention over configuration or coding. In other words, it simplifies the job of the developer.

It also comes with an extension to the Maven. This extension packages a microservice as a JAR, as opposed to a WAR file. Not only that, the JAR file even consists of an embedded Tomcat Web server. What does this mean? As long as you have a JRE, you can simply place the JAR file in the *classpath* and start the service. Spring Boot takes care of starting the Web server and mapping it to the service endpoints.

In other words, with Spring Boot, you no longer pre-install the Tomcat, no longer pre-provision the Tomcat and no longer co-host several applications in the same Tomcat. Instead, each application is self-sufficient with its own Web server, as shown in Figure 3. To summarise, the deployment unit consists of application code along with all the required libraries and Tomcat. Naturally, you are free to deploy the applications on the same machine or on multiple machines without worrying about setting up the server beforehand.

Let us build a Spring Boot microservice to demonstrate this.

UMS AddService

In the previous article in this series, we have developed a design for *AddService* as part of UMS. That design proposed a domain model, application layer, and infrastructure layer on the lines of the Onion model of domain-driven design. Though we can implement the proposed design completely on our own, why should we do it when a framework like Spring Boot offers some of those components out-of-the-box? After all, Spring and Spring Boot offer ready-to-use factories, proxy generators, weavers of responsibility chains, adapters, etc.

As you know, *AddService* offers only one functionality — to add a user to the system. In line with the contemporary style, let the *AddService* expose the functionality via REST